

The Grid Optimization Engine: Agentic Policy Search for Flexible Load Dispatch on a Calibrated Market Twin

A methodology white paper · gridcouncil project · August 2026

Abstract

Wholesale electricity markets clear at the marginal price, so a single new demand peak — a data center interconnecting, an EV fleet charging after work — can raise costs for every consumer on the grid. We present a working system that answers, on real market data, both halves of the question this poses to cities, utilities, and operators: *how much new load can the grid absorb before prices spike, and what should that load do about it?* The system combines (i) a **digital twin** of a wholesale market — a merit-order supply stack reconstructed by monotone regression from public day-ahead prices, load, and fuel-mix data, validated out of sample; (ii) a **scenario planner** that prices counterfactual load injections through the twin; (iii) a **flexibility optimizer** that dispatches deferrable compute and co-located storage, implemented in two policy classes — threshold rules (PFA) and a perfect-foresight linear program (DLA) that doubles as a provable bound; (iv) an **agentic policy-search loop** in which large language model (LLM) agents write, evaluate, and iteratively refine executable dispatch policies as ordinary code, following the hierarchical search architecture of Sommer et al. [1]; and (v) a **multi-actor coordination layer** that exposes the difference between selfish and coordinated flexibility, in the spirit of Lagrangian dual decomposition. We report three empirical studies: a faithful replication of [1] on a simulated residential battery system (best policy within 0.24 € of the 168-hour optimum; we additionally observe a divergent median trajectory under a smaller code model, amplifying the saturation-degradation effect the original authors describe); an out-of-sample twin calibration on NYISO data (correlation 0.88, MAE \$20.7/MWh); and an application of agentic policy search to a 500 MW data-center dispatch problem against the LP bound, where LLM-written policies closed 27% of the naive-to-optimal gap versus 55% for hand-written rules — an honest negative-leaning result that motivates our design decision to keep classical optimizers inside the agentic loop as both competitors and verifiers. All experiments are reproducible from the repository; every prompt, response, strategy, and score of every search run is archived verbatim.

1. Introduction

Electricity demand in the United States is rising for the first time in two decades, driven substantially by AI data centers and transport electrification. Because organized wholesale markets settle at the clearing price of the *most expensive* generator dispatched, demand growth concentrated in peak hours has a leveraged effect on consumer bills. Recent capacity auctions in constrained regions have cleared at or near administrative price caps, largely attributed to projected data-center load.

The central observation motivating this work is that the new load is not rigid. AI training jobs tolerate deferral on the order of hours to days [9, 10]; data centers increasingly co-locate storage; EV charging is overwhelmingly flexible overnight load. If that flexibility is dispatched against market conditions, new demand can be absorbed without setting a new marginal price. If it is not — or worse, if many flexible actors respond *identically* to the same price signal — flexibility can itself create new peaks (the herding effect we demonstrate in §6).

Two methodological commitments distinguish our system:

1. **Policies are code.** Control strategies are explicit, auditable Python classes — never weights inside an opaque model. This follows the argument of [1] that executable, human-readable policies are a precondition for deployment in safety-critical infrastructure, and stands in contrast to direct-LLM control, where the decision logic remains implicit in model parameters.

2. **The optimizer is also the verifier.** Wherever the dispatch problem is convex we retain the exact linear-programming solution, which serves three roles simultaneously: a deployable policy (model-predictive control), a provable lower bound for benchmarking learned and LLM-written policies, and the sound verifier of an LLM-Modulo arrangement [8] in which language-model agents propose and a non-AI system checks.

The remainder of the paper describes the modeling framework (§2), the digital twin and its calibration (§3), the scenario planner and flexibility optimizer (§4), the agentic policy-search architecture and its two experimental campaigns (§5), the multi-actor coordination layer (§6), the observability and live-operation design (§7), and limitations with future work (§8).

2. Modeling framework

We formulate every sequential decision problem in the system using the universal modeling framework of Powell [2, 3], which describes any such problem by five elements:

- **State** S_t : all information available for the decision at time t .
- **Decision** $x_t = X^{\pi}(S_t)$, produced by a policy X^{π} .
- **Exogenous information** W_{t+1} : what becomes known only after deciding.
- **Transition function** $S_{t+1} = S^M(S_t, x_t, W_{t+1})$.
- **Objective**: max (or min) over policies of the expected cumulative contribution $\sum_t C_t(S_t, x_t, W_{t+1})$.

The framework's taxonomy of the four fundamental policy classes — policy-function approximations (PFA), cost-function approximations (CFA), value-function approximations (VFA), and direct lookahead approximations (DLA) — is used *architecturally*: rather than committing to one class, the system implements members of several classes for the same problem and arbitrates among them by realized cost on the twin. In the present build, PFAs appear twice (hand-written threshold rules, and the LLM-written policies of §5), and the DLA appears as the perfect-foresight LP. The arbitration principle is that no class dominates every regime [3]; empirically (§5.4), the DLA dominates the convex dispatch problem — itself a finding the arbitration surfaces honestly.

Instantiation for the data-center problem. State: hour of day, baseline price, firm load, newly arriving deferrable compute, backlog and the age of its oldest element, battery state of charge. Decision: (flex_serve_mw, battery_mw) — how much deferred work to run this hour and how to move the battery. Exogenous: the price/net-load realization. Transition: FIFO backlog aging with a 24-hour deadline (overdue work is force-served by the environment, mirroring the constraint-enforcement-by-clipping convention of [1]); battery dynamics with per-leg efficiency $\sqrt{\eta_{rt}}$ so the round trip matches the rated efficiency. Objective: minimize the *added system dispatch cost* — the integral of the supply stack between the baseline and the with-load net demand (defined in §3) — rather than the private energy bill; the two are aligned but not identical, and the system-cost objective is what neutralizes price impact for all consumers.

3. The digital twin

3.1 Data

All market data are public. We use the NYISO open feeds via the `gridstatus` library [7]: day-ahead zonal locational marginal prices (LMPs; reference zone N.Y.C., all 11 zones retained for spatial visualization), actual system load, and the five-minute fuel mix aggregated hourly. The demo application uses a full year (Aug 2025–Aug 2026, 8,760 hours); the search experiments use a 75-day window with an 80/20 chronological train/test split so that every reported number is out of sample. Hourly average grid carbon intensity is computed from the fuel mix with standard per-fuel emission factors (natural gas 0.42, dual fuel 0.46, other fossil 0.90 tCO₂/MWh; nuclear, hydro, wind ~ 0), giving every cost figure a parallel carbon figure. The architecture isolates the data layer behind a

two-function adapter, so a PJM (or any ISO) feed substitutes without touching the rest of the system; PJM's Data Miner requires a subscription key and is stubbed pending one.

3.2 Merit-order reconstruction by monotone regression

The twin models the day-ahead price as a function of *net load* — system load minus must-run, zero-marginal-cost generation (nuclear, hydro, wind, other renewables), which displaces the thermal stack:

$$p_t = f(L_t - M_t) + a_{\{h(t)\}} + \epsilon_t$$

where f is the **supply stack**, constrained to be non-decreasing (a merit order cannot slope down), a_h is a small additive hour-of-day adjustment absorbing systematic diurnal effects not captured by net load (e.g., gas-unit commitment patterns), and ϵ is noise. We fit f by **isotonic regression** on the training split — the natural nonparametric estimator under a monotonicity constraint, requiring no assumed functional form and no proprietary heat-rate data — and clip extrapolation beyond observed net load to the stack's end segments, extended linearly with the last segment's slope so that scarcity pricing continues to steepen out of sample.

The stack yields a **system dispatch cost** functional: $C(x) = \int_0^x f(u) du$, evaluated by trapezoid integration on the fitted breakpoints. The *added system cost* of any injected load profile d_t is $\sum_t [C(b_t + d_t) - C(b_t)]$, where b_t is baseline net load. Because f is non-decreasing, C is convex — the property that makes the DLA of \$4 an exact linear program and its optimum a provable bound.

3.3 Calibration and validation

Calibration is the credibility anchor: the twin must track prices it has not seen before its counterfactuals mean anything. On the held-out final 20% of the 75-day window (360 hours), the twin achieves Pearson correlation 0.88 and MAE \$20.7/MWh against realized day-ahead prices, with the fit degrading exactly where one would expect — the sharpest scarcity hours — and reported as such in the application (the calibration scatter, time-series overlay, and residual profile are a first-class exhibit, not an appendix). We do not tune on the test window; the search experiments and all headline numbers use the same held-out period.

4. Scenario planner and flexibility optimizer

4.1 Counterfactual pricing

A scenario is a load injection: a flat or shaped d_t added at the reference zone (spatial siting across zones is future work; the zonal price data to support it is already collected). The planner reprices the horizon through the twin, $\hat{p}_t = f(b_t + d_t) + a_{\{h(t)\}}$, and reports the full distributional consequence: the new price duration curve, the peak-hour price impact, the added system cost, the injector's own energy bill, and the *consumer bill impact* — the price uplift applied to all coincident system load, which is the number that matters to a city. For the benchmark scenario (a rigid 500 MW data center on the held-out fortnight), the twin attributes a peak clearing-price impact of +\$196/MWh and a consumer-bill impact of ~\$76M over two weeks — the "do nothing" case against which all interventions are scored.

4.2 Two policy classes for the same lever

The intervention is the flexibility the load already has: a deferrable fraction of compute (50% in the benchmark configuration, 24 h deadline) and co-located storage (400 MWh / 100 MW, 88% round-trip). We implement the dispatch problem twice:

- **PFA (threshold rules):** defer when the current price exceeds a rolling quantile threshold, catch up below a lower quantile, with deadline-forced serving; battery charges below/discharges above symmetric thresholds. Transparent, myopic, cheap.

- **DLA (lookahead LP):** minimize the convex stack cost over the full horizon subject to backlog conservation with deadlines, battery dynamics, and power limits. The convex separable objective is represented by segment-wise linearization of C , making the problem an LP solved by HiGHS. Under perfect foresight of the baseline this is the *optimal* dispatch, so its cost is simultaneously the best achievable and the yardstick for everything else.

On the benchmark scenario the added system cost falls from \$14.88M (naive, inflexible) to \$13.30M (PFA) to \$12.02M (DLA); the data center's own bill drops 22%, and the consumer-bill impact falls by a third. Because the engine can re-solve the DLA under varied storage configurations, it also answers the *sizing* question — the storage (250 MW / 1 GWh in the benchmark) at which the marginal price impact of the interconnection is brought under a target threshold (+\$56/MWh peak impact in the benchmark) — turning the counterfactual into an interconnection strategy.

5. Agentic policy search

5.1 Architecture

The search layer follows the three-level hierarchy of Sommer et al. [1]:

- **Level 1 — execution:** the candidate policy runs closed-loop in the evaluation environment (the simulated home of §5.3 or the twin-backed dispatch environment of §5.4), which enforces physical limits by clipping and returns scalar metrics.
- **Level 2 — implementation:** a code-generating LLM ("strategy writer") converts a natural-language task description into a complete policy class conforming to a fixed interface. Output passes a post-processing function Φ (markdown stripping, class extraction) and is executed in a sandboxed subprocess. A runtime failure triggers a repair prompt carrying the traceback and the failed code, for up to five attempts, after which the round restarts — the error-recovery loop of [1].
- **Level 3 — meta-policy:** a second LLM ("coach") observes only aggregate outcome metrics — never raw time series, a deliberate information bottleneck that [1] motivates as reward-hacking prevention — plus the current implementation, and emits the next task description. It follows the greedy rule of [1]: after an improvement, *refine* ("suggest ONE specific improvement"); otherwise *explore* ("propose a novel approach"); with an automatic switch to exploration when no improvement exceeds a threshold over a stagnation window (3 rounds).

The loop always retains the best-scoring policy found (the search is an argmax over its own history, so late-round degradation cannot damage the deliverable). Agents communicate through a filesystem protocol — each episode pauses on a `pending_prompt.txt`, any worker process serves it with a completion in `response.txt`, and a driver advances the state machine — which makes the orchestration LLM-agnostic, resumable after interruption, and trivially observable (§7).

5.2 Relation to prior architectures

The design difference from tool-integrated monolithic reasoning loops is the same one argued in [1]: decoupling policy planning (coach) from implementation (writer) from evaluation (simulator) yields structured validation and reproducible, simulation-scored candidates. Our multi-agent framing — a global coordinator allocating work to local actors, with a non-AI environment closing the loop — also mirrors the hierarchical workload-manager/local-agent decomposition used by carbon-aware data-center scheduling systems [4], though those systems learn continuous policies by multi-agent reinforcement learning (MAT [6]) where we search over discrete programs.

5.3 Campaign 1: residential battery (replication of [1])

We replicated the experimental scenario of [1] end to end: a 7-day, hourly-resolution home energy system (10 kWh battery at 90% round-trip efficiency, 5 kWp PV, double-peak household load, volatile buy price with mean 0.35 €/kWh and 10% volatility, fixed 0.08 €/kWh feed-in), with the paper's verbatim prompt templates, interface signature, repair loop, and greedy meta-policy. Because the paper's stochastic processes are under-specified, we calibrated free shape parameters against its three published benchmark costs; our deterministic benchmarks land within 0.35 € (no-battery 10.64 vs 10.70; perfect-foresight optimum -6.32 vs -6.67; steady-state -5.55 vs -5.20).

Results (10 episodes × 10 iterations, Claude Haiku 4.5 as both writer and coach). The headline claim of [1] replicates: every episode found a profitable policy, 9 of 10 within 1.8 € of the optimum, and the best policy (-6.08 €, within 0.24 € of the bound, i.e. 96% of the achievable gap) exhibits the same full-daily-cycle, self-consumption-plus-arbitrage behavior as the paper's exemplar. The paper's *median convergence*, however, did not replicate: our median cost diverges after iteration 2 (-2.9 € → +20.6 €), because with strong first-round policies the greedy rule rarely registers improvement, the coach explores nearly permanently, and each exploration instruction pushes the writer toward increasingly elaborate implementations whose unit errors (e.g., €/kWh thresholds three orders of magnitude off) buy heavily from the grid. This is the saturation-degradation phenomenon §9 of [1] describes, amplified by a smaller code model — and it is harmless to the deliverable precisely because the search keeps the best-so-far. The repair loop resolved all 10 runtime failures encountered across 100 generated policies.

5.4 Campaign 2: data-center dispatch against a provable bound

We then pointed the identical search machinery at the real-data problem of §4, with the LP optimum as the known bound — a sharper experimental design than Campaign 1, because the environment is deterministic (all variance is the LLM's) and the bound is exact. The policy interface adds a deadline-carrying backlog to the state; the objective is added system cost.

Results (5 episodes, stopped at 5–6 iterations on stagnation). All 25+ generated policies ran (crash rate near zero; two repair events), and every episode beat the naive baseline immediately. The best LLM-written policy reached \$14.10M added system cost versus \$14.88M naive, \$13.30M hand-written PFA, and \$12.02M LP — closing 27% of the naive-to-optimal gap where the hand rules close 55%. The meta-level again failed to drive improvement beyond early rounds. We report this as the paper's honest finding: **for convex, foresight-friendly dispatch, classical optimization dominates, and the correct role of the LLM layer is proposal, explanation, and search over regimes the LP cannot express** (non-convex tariffs, unmodeled constraints, novel strategy structure) — not competition with the LP on its own ground. This is the empirical justification for the arbitration architecture of §2.

5.5 Self-explaining policies

The policy interface (v2) requires each action to return a one-sentence *reason* citing the numbers that drove it ("deferring: price \$87 exceeds the 75th-percentile threshold \$52; backlog 9 h from deadline"). Reasons are generated by the same code that acts — they are the policy's own branch logic verbalized, not post-hoc rationalization by a separate model — and are recorded per hour into the best-policy trace, rendering as a decision ledger in replays. This operationalizes the interpretability argument that motivates code-as-policy in [1].

6. Multi-actor coordination

The flagship demonstration composes two flexible actors on one twin: the 500 MW data center and an aggregated fleet of battery-equipped households (parameterized by an opt-in count; 50,000 homes ~ 250 MW / 675 MWh), against a real spike episode from the held-out data. Three regimes are computed:

1. **Uncoordinated (price-taker herding):** each actor independently minimizes its own bill against the *baseline* price forecast — the realistic default, since retail actors are price-takers. Solved as separate linear programs with linear (price-vector) objectives, then jointly repriced through the stack. The characteristic failure appears:

all storage discharges into the same peak and recharges in the same trough, and the data center dumps its backlog there too, manufacturing a *secondary peak* — the rebound effect documented in the demand-response literature.

2. **Negotiated (dual decomposition):** a coordinator iterates: broadcast current prices -> actors re-solve their price-taker LPs -> reprice the joint dispatch through the stack -> damp and repeat. This is a damped price-signal iteration in the spirit of Lagrangian dual decomposition of the joint problem, and it visibly converges within a handful of rounds — the "negotiation" rendered as an animation of the flattening net-load curve.

3. **Coordinated bound:** the joint LP over both actors' variables against the shared convex stack cost — what a perfect coordinator achieves.

The exhibit reports peak shaved, rebound magnitude eliminated, and the per-actor ledger (consumer uplift avoided, fleet revenue per home, data-center savings) for each regime. Methodologically, the point is that the *gap between regimes 1 and 3 is the value of coordination*, and regime 2 shows that most of it is recoverable through prices alone — the mechanism a real aggregator or DSO could operate.

7. Observability and live operation

Every search run is a complete, replayable record: per-round scores and metrics (`state.json`), every generated strategy as a file, an append-only event log (scores, coach verdicts with excerpts, crashes, spike injections), and — as of v2 — verbatim transcripts of *every* prompt and response exchanged, archived by the driver before consumption. The demonstration application replays recorded runs round by round (instruction -> strategy -> score -> coach's verdict, with code diffs), exposes all prompt templates, initial data, and logs in a transparency view, and runs **live**: a worker process (Anthropic API or CLI backend; an OpenAI-compatible local backend is planned) serves pending prompts while the interface tails the event stream. A mid-run perturbation tool injects a demand spike or price surge into the evaluation environment *while the search is running* — subsequent rounds are scored in the changed world, previously recorded benchmarks are flagged stale, and the coach's reaction to the regression is observable in its next instruction. This turns the adaptivity claim of the underlying method [1] into a demonstrable, interactive event rather than an offline assertion.

8. Limitations and future work

Twin fidelity. The twin is a single-zone, average-hourly-adjustment model: no transmission constraints, unit commitment, ramping, or weather covariates; its scarcity-hour errors are its largest. Zonal congestion modeling (the data is collected), weather features, and real-time (5-minute) price layers are the natural next increments. **Marginal vs average carbon.** Carbon figures use average intensity; dispatch-marginal emissions, or a full carbon-emission-flow treatment with nodal carbon intensity [5], would be required for rigorous carbon-aware scheduling claims of the kind made in [4]. **Search effectiveness.** Both campaigns show the meta-level failing to improve on strong early candidates — consistent with [1]'s own discussion of its myopic selection rule — and our stagnation-triggered exploration did not rescue it. Richer meta-state (e.g., which hours lose the most money vs the bound), best-of-N candidate selection per round, and model selection at the meta level (comparing writer models on the same problem — the experiment the planned local-inference backend enables) are the direct follow-ups. **Coordination realism.** The household fleet is an aggregate; per-device modeling on public building-stock load profiles (e.g., NREL ResStock [11]) and incentive-compatibility of the negotiated settlement are open. **Actors as agents.** Replacing the two hand-modeled actors of §6 with independently searched, communicating policies — a workload-manager/local-agent structure as in [4] — is the multi-agent extension of Campaign 2.

9. Conclusion

The system demonstrates an end-to-end methodology for the grid's demand-growth question: reconstruct the market from public data and validate out of sample; price the counterfactual; dispatch the flexibility the new load already has, with the exact optimizer retained as bound and verifier; let LLM agents search the policy space as auditable code, scored by simulation, with every step recorded; and expose the coordination gap between selfish and orchestrated flexibility. The empirical results are deliberately mixed and honestly reported: agentic policy search reliably produces working, near-optimal policies for the stochastic household problem it was designed for, while classical optimization dominates the convex industrial dispatch problem — which is precisely why the architecture arbitrates across policy classes instead of betting on one.

References

- [1] A. Sommer, P. Bazan, B. Babaeian, J. Fellerer, W. B. Powell, and R. German, "Adaptive Self-Improvement for Smarter Energy Systems using Agentic Policy Search," Friedrich-Alexander-Universität Erlangen-Nürnberg / Princeton University.
- [2] W. B. Powell, *Reinforcement Learning and Stochastic Optimization: A Unified Framework for Sequential Decisions*, Wiley, 2022.
- [3] W. B. Powell and S. Meisel, "Tutorial on Stochastic Optimization in Energy — Part II: An Energy Storage Illustration," *IEEE Transactions on Power Systems*, vol. 31, no. 2, pp. 1468–1475, 2016.
- [4] H. Lee, P. Prabawa, D.-H. Choi, and J. Kim, "Hierarchical Multi-Agent Reinforcement Learning for Carbon-Aware AI Data Centers in Power Distribution Systems," arXiv:2607.03324, 2026.
- [5] C. Kang et al., "Carbon Emission Flow from Generation to Demand: A Network-Based Model," *IEEE Transactions on Smart Grid*, vol. 6, no. 5, pp. 2386–2394, 2015.
- [6] M. Wen, J. G. Kuba, R. Lin, W. Zhang, Y. Wen, J. Wang, and Y. Yang, "Multi-Agent Reinforcement Learning is a Sequence Modeling Problem," *Advances in Neural Information Processing Systems* (NeurIPS), 2022.
- [7] gridstatus: open-source Python library for ISO/RTO market data. <https://github.com/gridstatus/gridstatus>
- [8] S. Kambhampati, K. Valmeekam, L. Guan, M. Verma, K. Stechly, S. Bhambri, L. Saldyt, and A. Murthy, "LLMs Can't Plan, But Can Help Planning in LLM-Modulo Frameworks," *Proc. ICML*, 2024.
- [9] P. Wiesner, I. Behnke, D. Scheinert, K. Gontarska, and L. Thamsen, "Let's Wait Awhile: How Temporal Workload Shifting Can Reduce Carbon Emissions in the Cloud," *Proc. Middleware*, 2021.
- [10] A. Radovanović et al., "Carbon-Aware Computing for Datacenters," *IEEE Transactions on Power Systems*, vol. 38, no. 2, pp. 1270–1280, 2023.
- [11] NREL, "End-Use Load Profiles for the U.S. Building Stock" (ResStock), National Renewable Energy Laboratory, 2021.
- [12] W. Kersting, "Radial Distribution Test Feeders," *IEEE Transactions on Power Systems*, vol. 6, no. 3, pp. 975–985, 1991.